

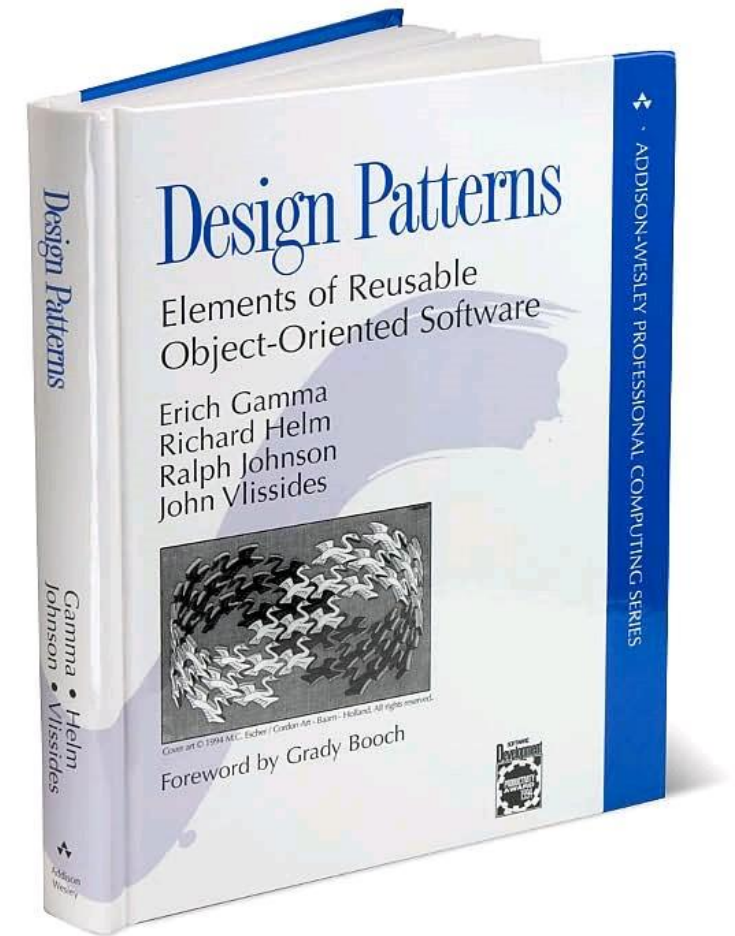
CSC3209 Software architecture and Design Patterns

Lecture13: Design Patterns Structural Patterns

Subject Lecturer: Dr. Muhammed Basheer Jasser

References

- Materials from
 - Gamma, Erich. Design patterns: elements of reusable object-oriented software. Pearson Education India, 1995.



Lecture Outline

- What is a Design Pattern
- Design Pattern Catalogue
 - Creational Patterns
 - **Structural Patterns**
 - Behavioural Patterns

Structural Patterns

- Structural patterns are concerned with how classes and objects are composed to form larger structures.
- Structural class patterns use inheritance to compose interfaces or implementations.

Structural Patterns

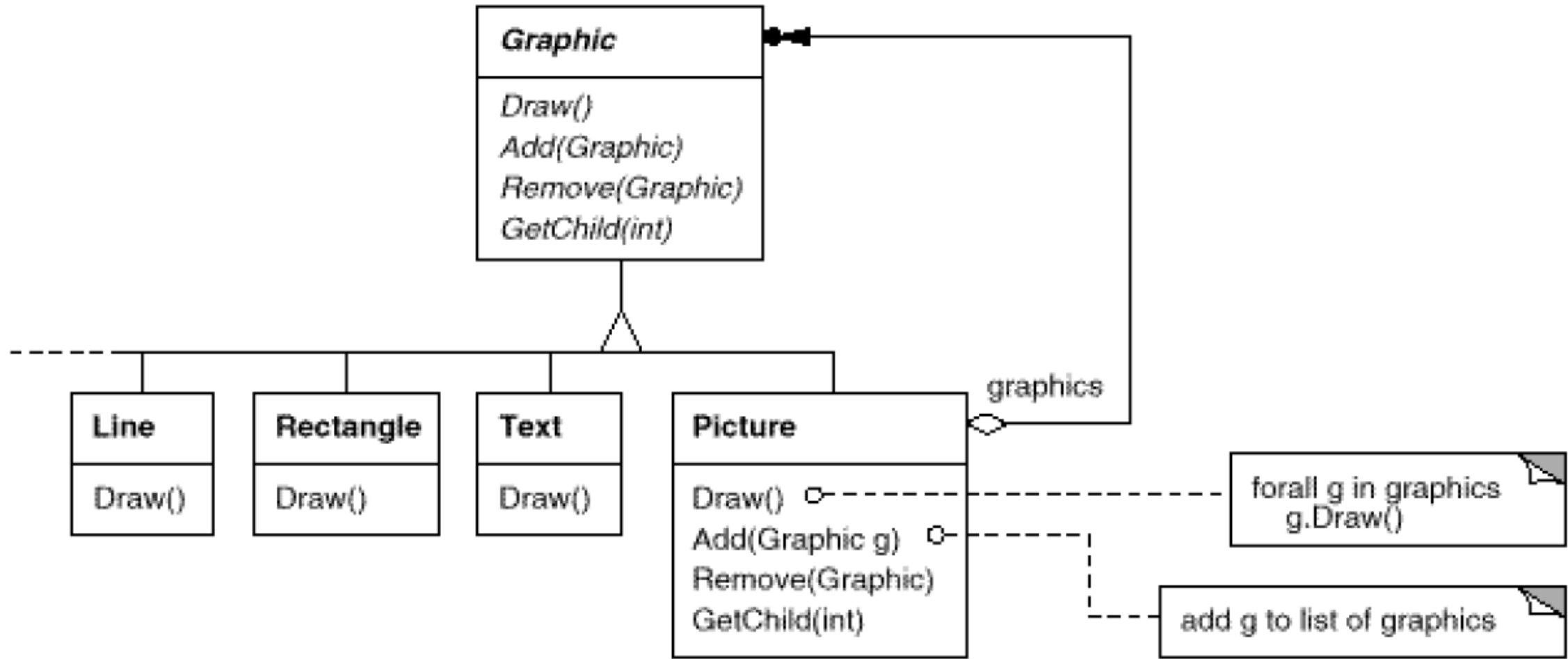
1. Adapter
2. Bridge
- 3. Composite**
4. Decorator
- 5. Façade**
6. Flyweight
7. Proxy

Composite : Structural Pattern

- **Intent:** Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.
- **Motivation(1):**
 - Graphics applications like drawing editors and schematic capture systems let users build complex diagrams out of simple components. The user can group components to form larger components, which in turn can be grouped to form still larger components.
 - A simple implementation could define classes for graphical primitives such as Text and Lines plus other classes that act as containers for these primitives.
 - But there's a problem with this approach: Code that uses these classes must treat primitive and container objects differently, even if most of the time the user treats them identically. Having to distinguish these objects makes the application more complex. The Composite pattern describes how to use recursive composition so that clients don't have to make this distinction.

Composite : Structural Pattern

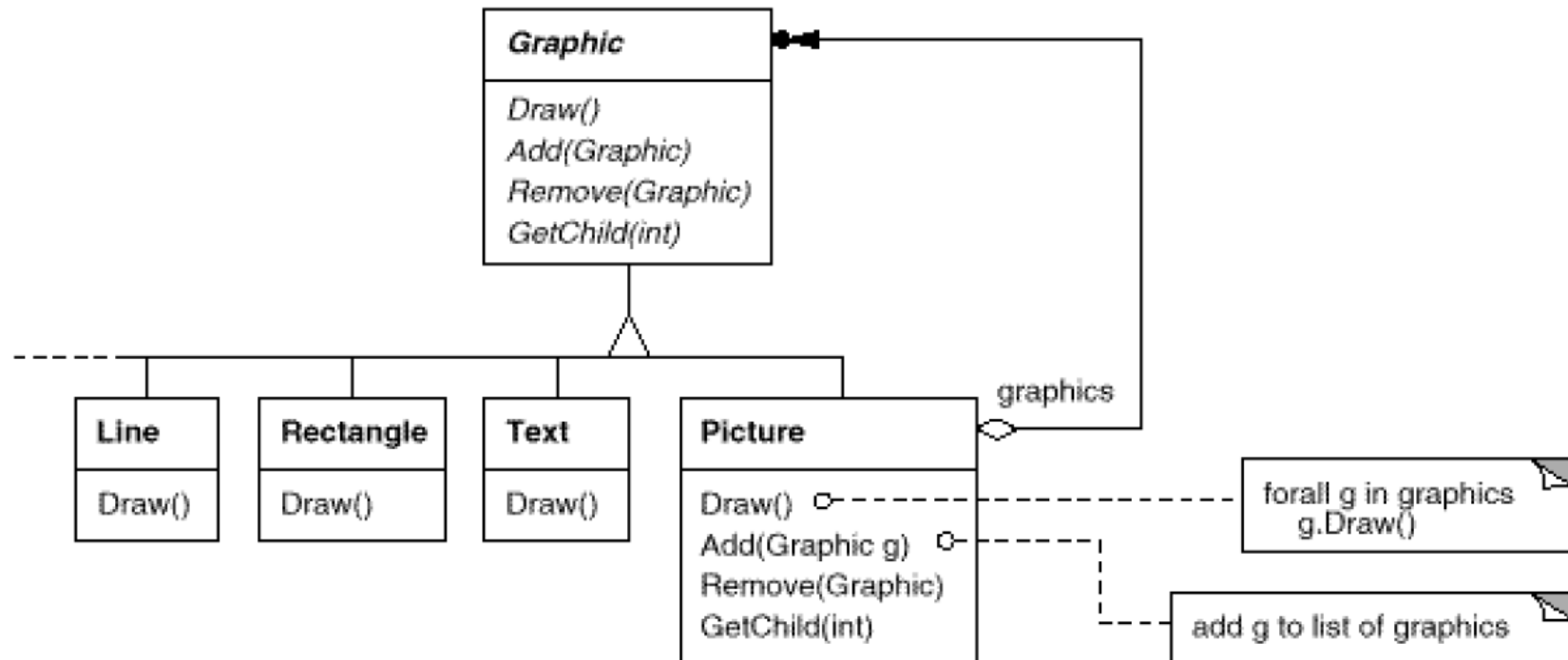
- Motivation(2):



Composite : Structural Pattern

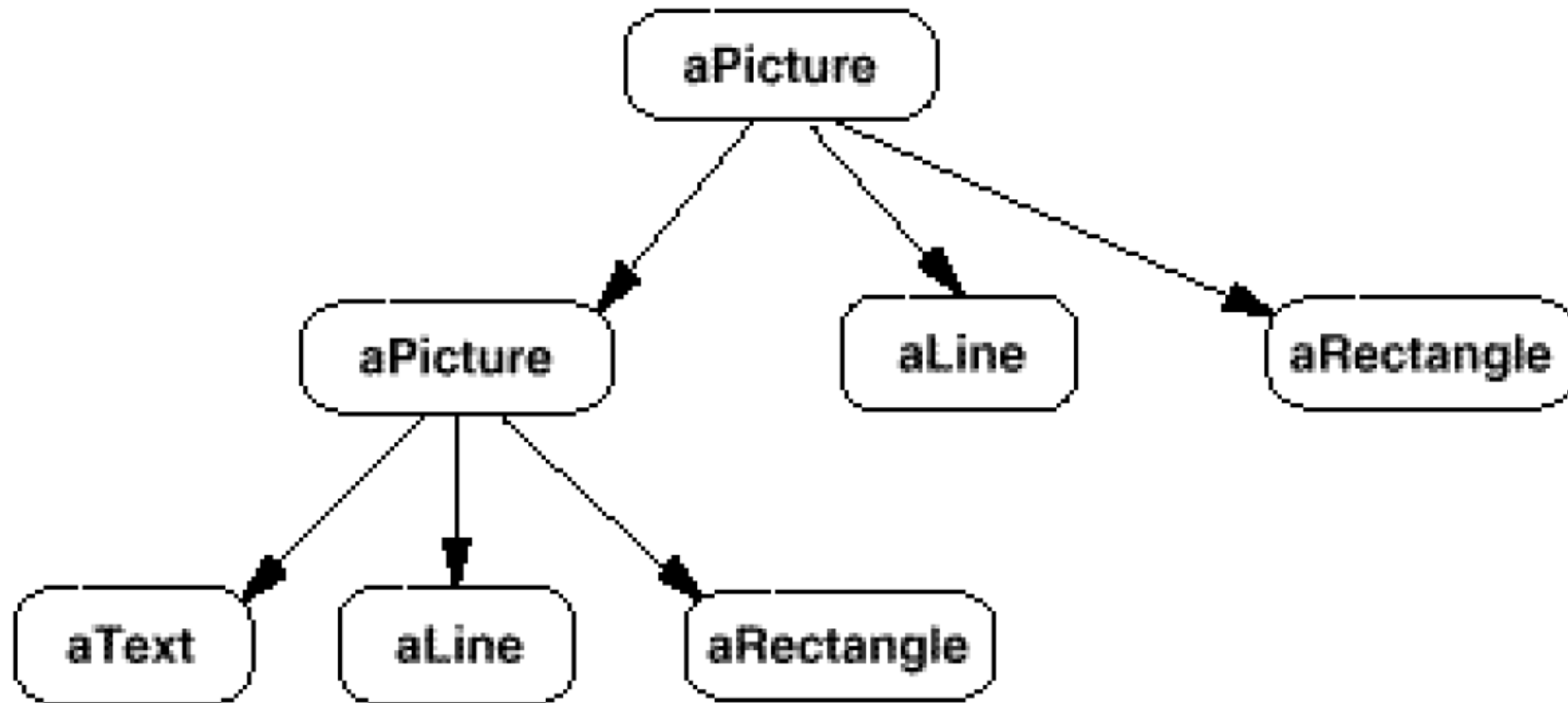
- **Motivation(3):**

- The **key** to the Composite pattern is an **abstract class** that **represents** both **primitives** and their **containers**. For the **graphics system**, this class is **Graphic**. **Graphic declares operations** like **Draw** that are **specific to graphical objects**.
- It also **declares operations** that **all composite objects share**, such as **operations** for **accessing** and managing its **children**.
- The **subclasses** **Line**, **Rectangle**, and **Text** define **primitive** graphical **objects**. These classes **implement Draw** to draw lines, rectangles, and text, respectively. Since primitive graphics have no child graphics, none of these subclasses implements child-related operations.



Composite : Structural Pattern

- **Motivation(4):**
- The **Picture class** defines an **aggregate** of **Graphic objects**. Picture **implements Draw** to call Draw on its children, and it implements child-related operations accordingly. Because the Picture interface conforms to the Graphic interface, **Picture objects can compose other Pictures recursively**.
- The following diagram shows a typical composite object structure of recursively composed Graphic objects:

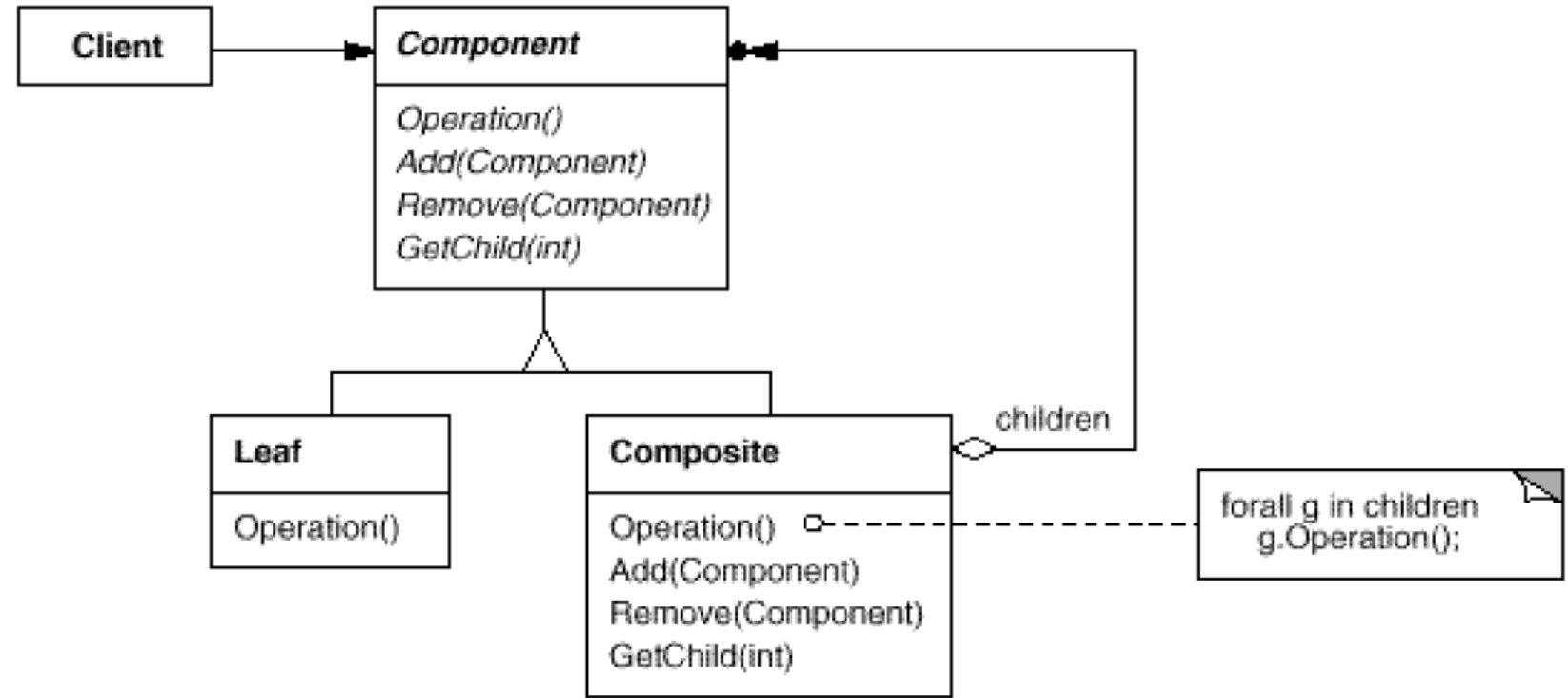


Composite : Structural Pattern

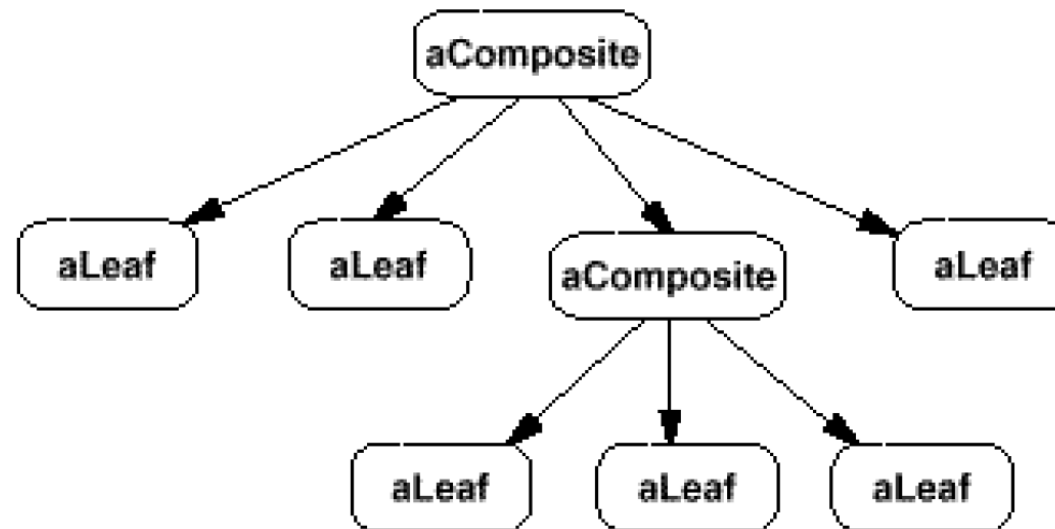
- **Applicability:** Use the Composite pattern when
 - you want to represent part-whole hierarchies of objects.
 - you want clients to be able to ignore the difference between compositions of objects and individual objects. Clients will treat all objects in the composite structure uniformly.

Composite : Structural Pattern

- Structure:



- A typical Composite object structure might look like this:



Composite : Structural Pattern

- **Participants:**
 - **Component** (Graphic)
 - declares the **interface** for **objects** in the composition.
 - **implements default behavior** for the interface common to all classes, as appropriate.
 - declares an interface for accessing and managing its child components.
 - **Leaf** (Rectangle, Line, Text, etc.)
 - represents **leaf objects** in the composition. A leaf has **no children**.
 - defines behavior for **primitive objects** in the composition.
 - **Composite** (Picture)
 - defines behavior for **components having children**.
 - stores child components.
 - **implements child-related operations** in the **Component** interface.
- **Client:**
 - manipulates **objects** in the composition through the **Component interface**.

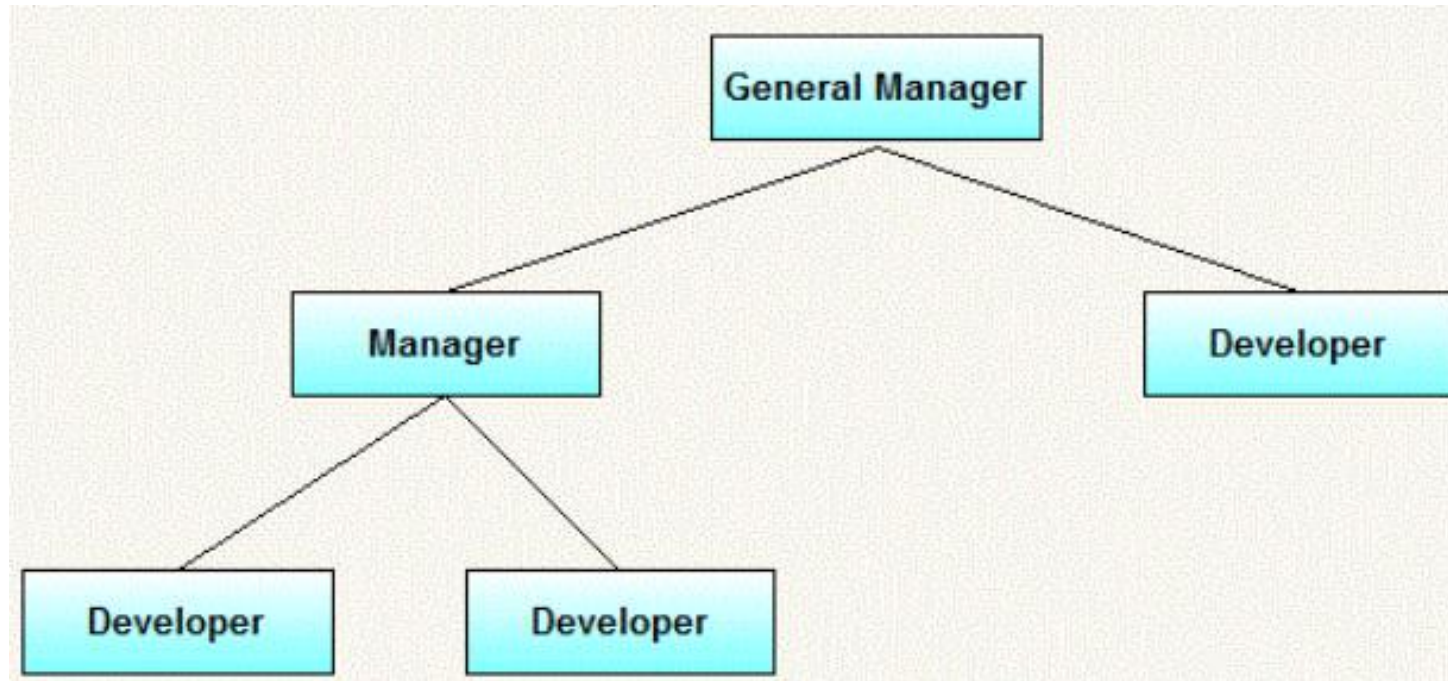
Composite : Structural Pattern

- **Collaborations:**

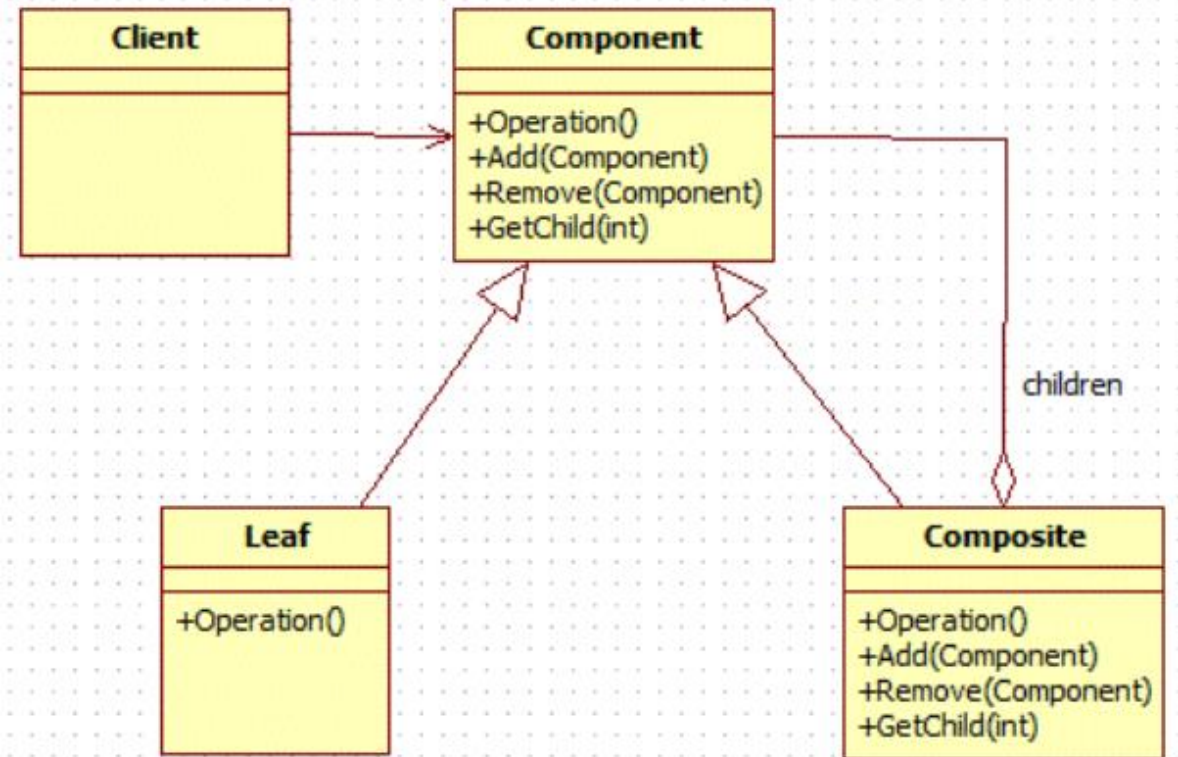
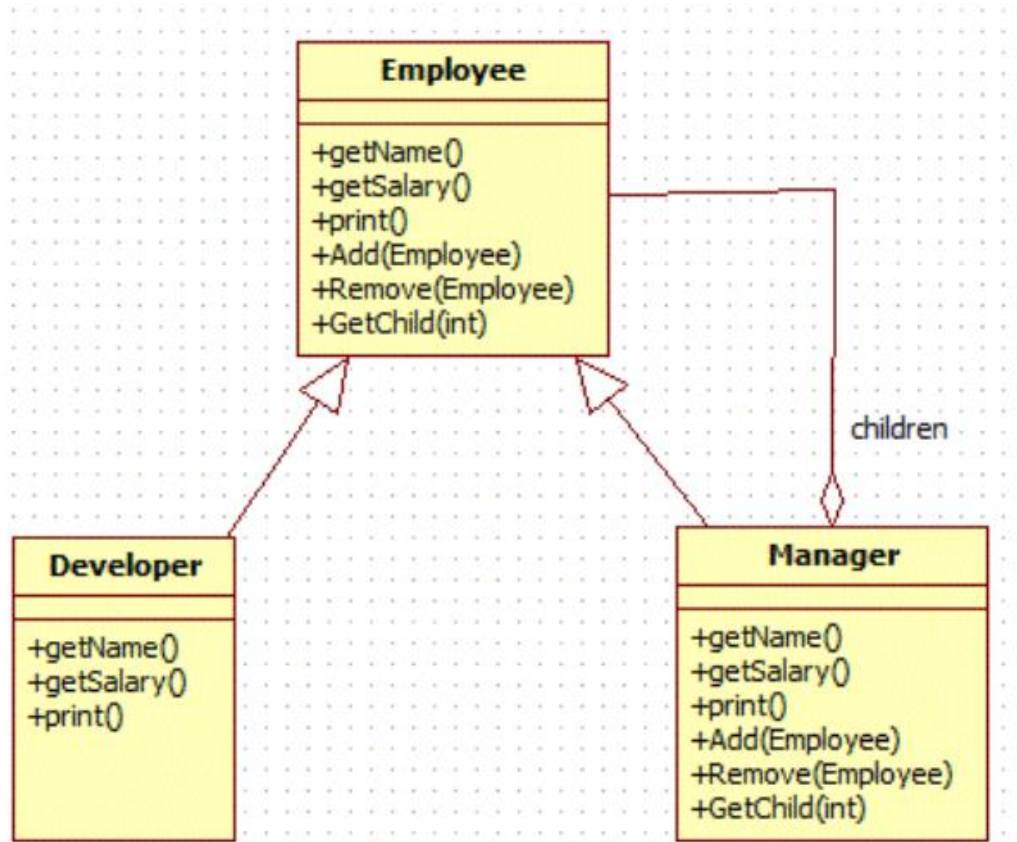
- Clients use the Component class interface to interact with objects in the composite structure.
- If the recipient is a Leaf, then the request is handled directly.
- If the recipient is a Composite, then it usually forwards requests to its child components, possibly performing additional operations before and/or after forwarding.

Composite : Structural Pattern (Example)

- In a small organization, there are 5 employees. At top position, there is one general manager.
- Under the general manager, there are two employees, one is manager and the other is developer.
- The manager has two developers that work under him.
- We want to print the names and salaries of all employees from top to bottom.

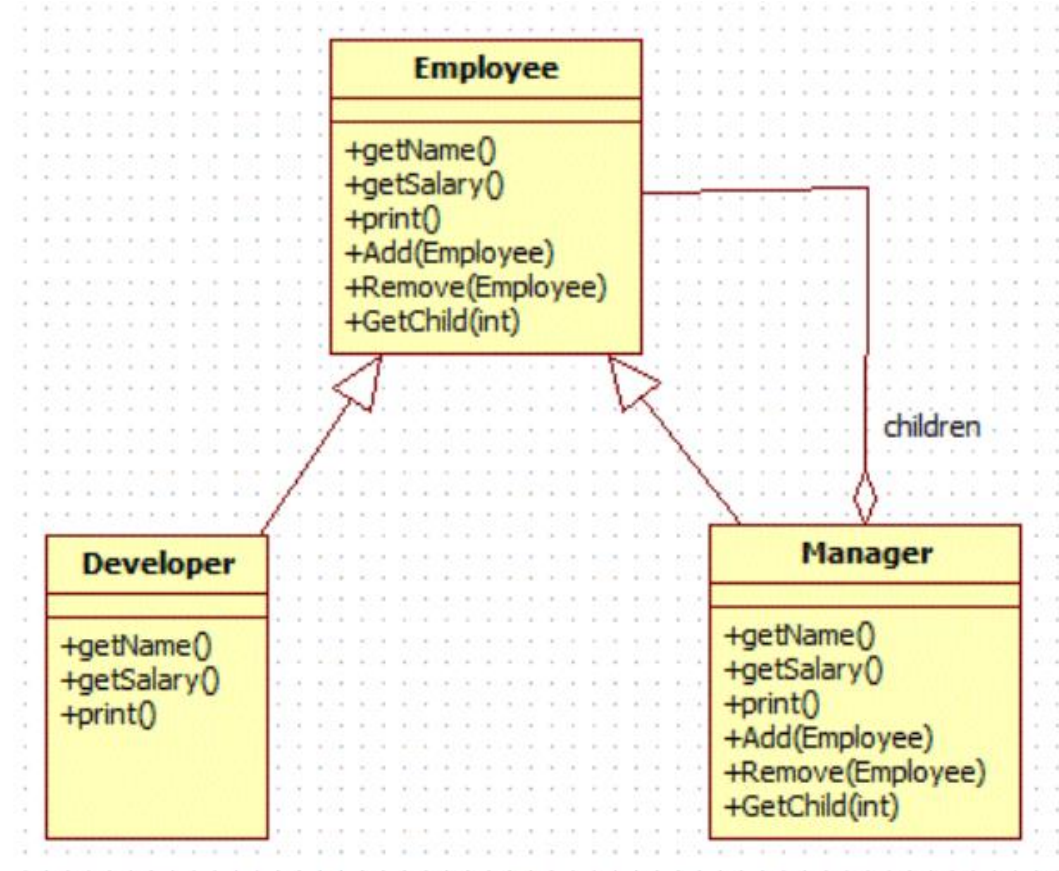


Composite : Structural Pattern (Example)



Composite : Structural Pattern (Example)

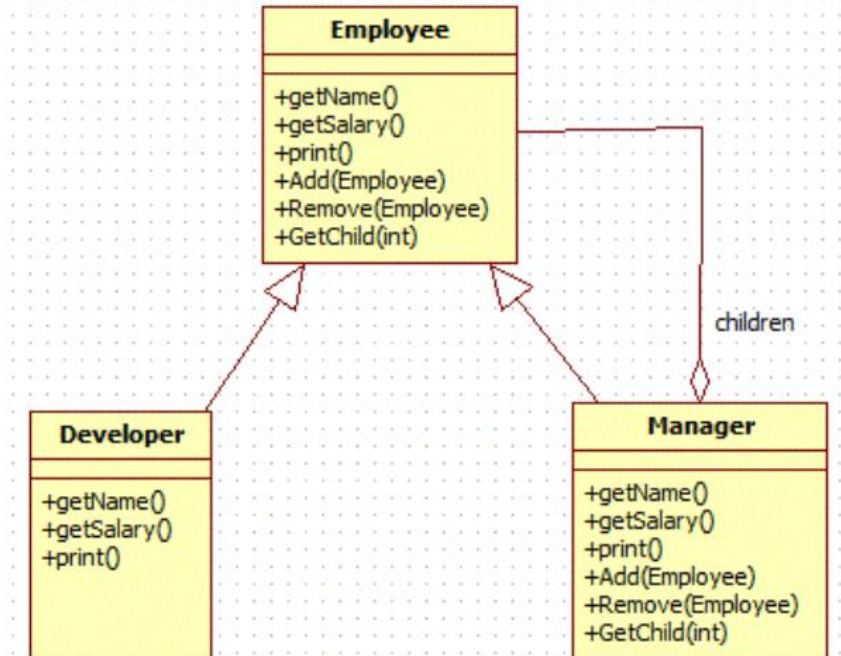
```
1 package CompositeExample;
2
3 public interface Employee {
4
5     public void add(Employee employee);
6     public void remove(Employee employee);
7     public Employee getChild(int i);
8     public String getName();
9     public double getSalary();
10    public void print();
11 }
```



Composite : Structural Pattern (Example)

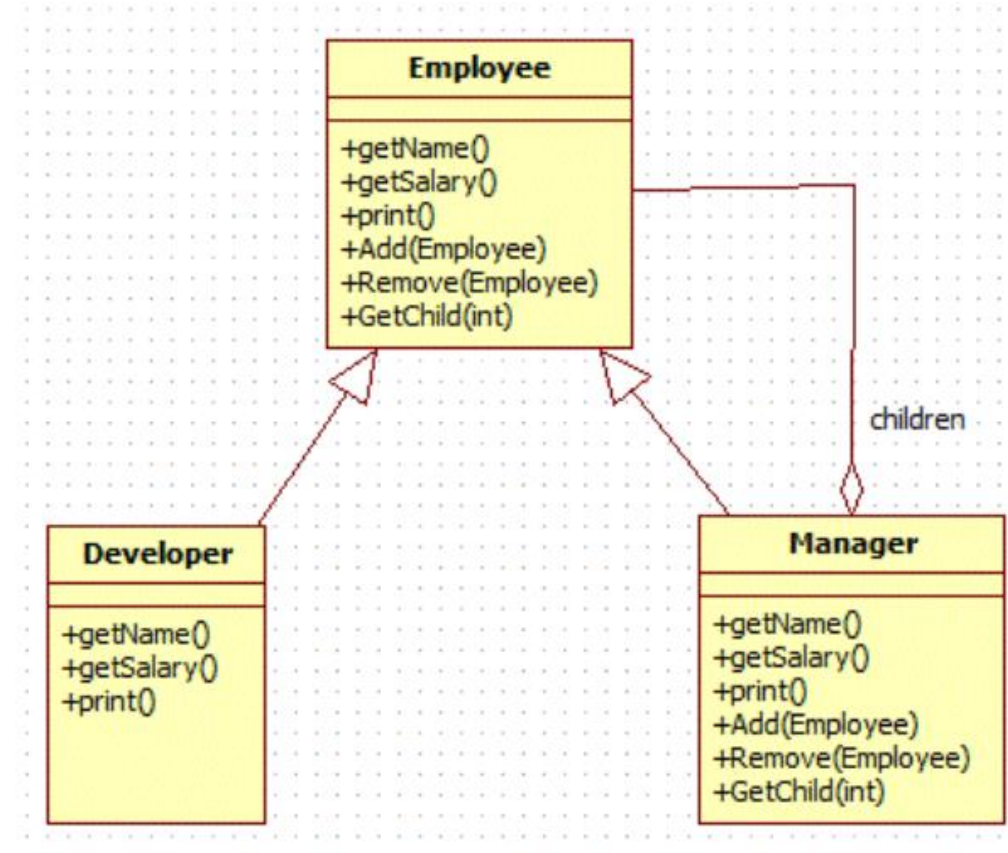
```
1 package CompositeExample;
2 import java.util.ArrayList;
3 import java.util.Iterator;
4 import java.util.List;
5
6 //The composite
7 public class Manager implements Employee{
8     private String name;
9     private double salary;
10 public Manager(String name,double salary){
11     this.name = name;
12     this.salary = salary;
13 }
14
15 List<Employee> employees = new ArrayList<Employee>();
16 public void add(Employee employee) {
17     employees.add(employee);
18 }
19
20 public Employee getChild(int i) {
21     return employees.get(i);
22 }
23
24 public String getName() {
25     return name;
26 }
27
28 public double getSalary() {
29     return salary;
30 }
31 }
```

```
32 public void print() {
33     System.out.println("-----");
34     System.out.println("Name =" + getName());
35     System.out.println("Salary =" + getSalary());
36     System.out.println("-----");
37
38     Iterator<Employee> employeeIterator = employees.iterator();
39     while(employeeIterator.hasNext()){
40         Employee employee = employeeIterator.next();
41         employee.print();
42     }
43 }
44
45 public void remove(Employee employee) {
46     employees.remove(employee);
47 }
48 }
```



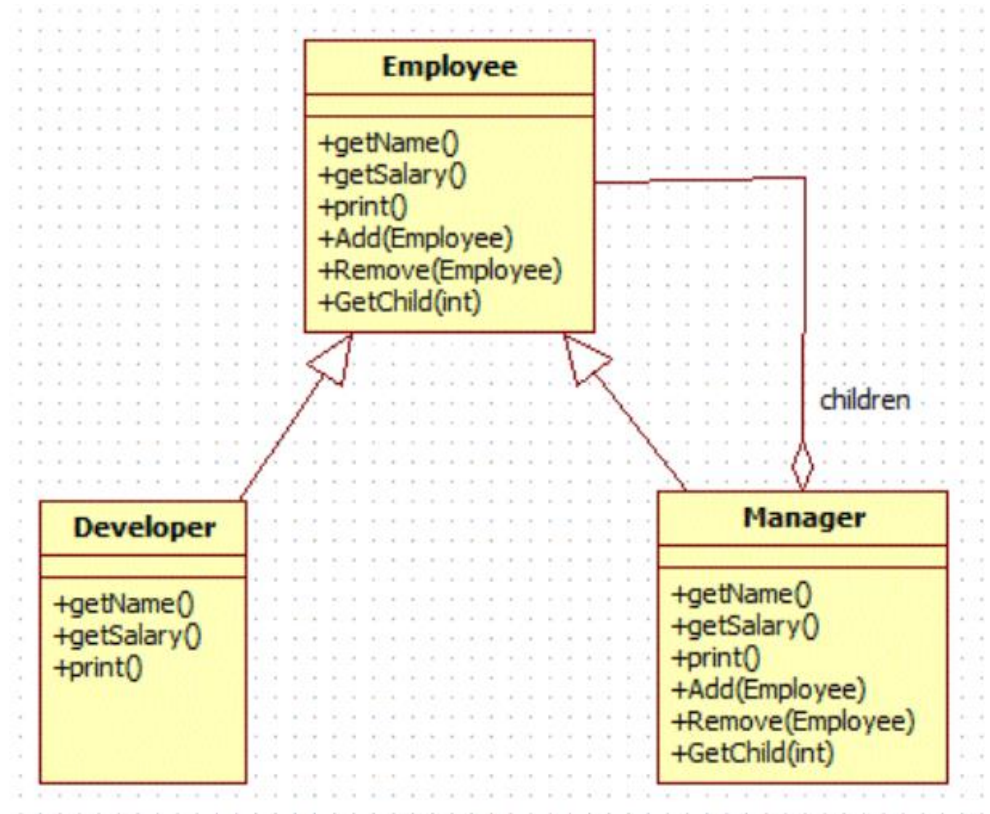
Composite : Structural Pattern (Example)

```
1 package CompositeExample;
2 //The leaf
3 public class Developer implements Employee{
4     private String name;
5     private double salary;
6
7     public Developer(String name,double salary){
8         this.name = name;
9         this.salary = salary;
10    }
11    public void add(Employee employee) {
12        //this is leaf node so this method is not applicable to this class.
13    }
14
15    public Employee getChild(int i) {
16        //this is leaf node so this method is not applicable to this class.
17        return null;
18    }
19
20    public String getName() {
21        return name;
22    }
23    public double getSalary() {
24        return salary;
25    }
26    public void print() {
27        System.out.println("-----");
28        System.out.println("Name =" + getName());
29        System.out.println("Salary =" + getSalary());
30        System.out.println("-----");
31    }
32    public void remove(Employee employee) {
33        //this is leaf node so this method is not applicable to this class.
34    }
35 }
```



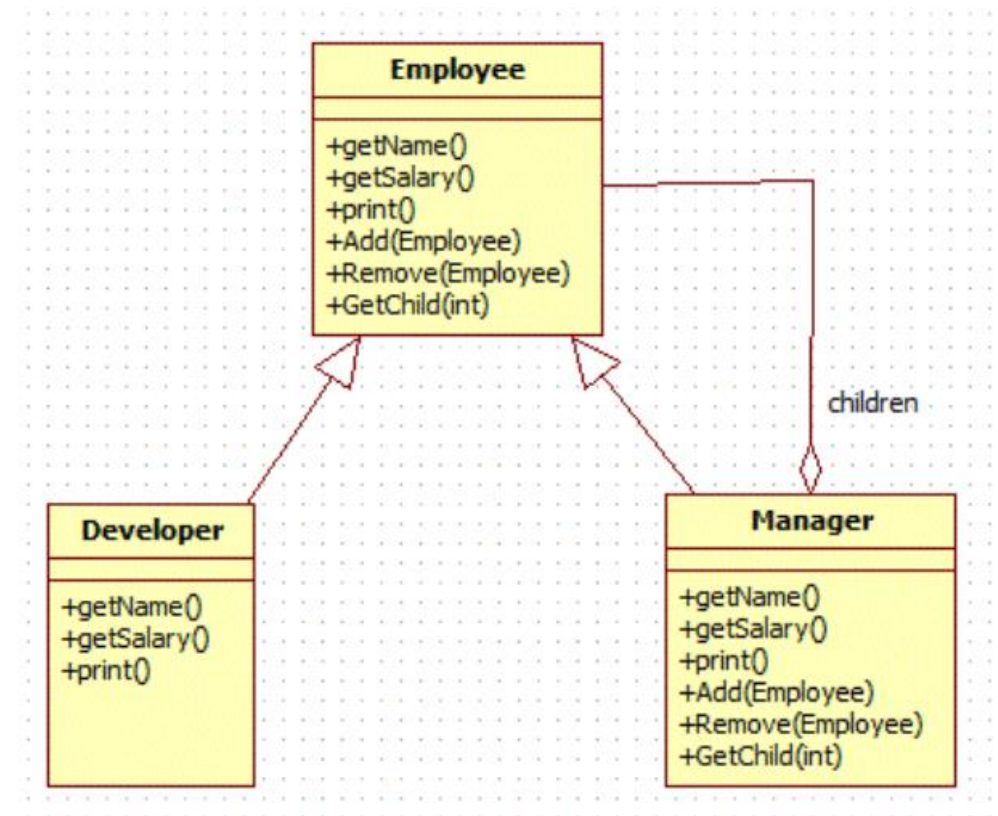
Composite : Structural Pattern (Example)

```
1 package CompositeExample;
2 public class CompositeDesignPatternMain {
3     public static void main(String[] args) {
4         Employee emp1=new Developer("Mohammad", 3500);
5         Employee emp2=new Developer("Lee", 4000);
6         Employee manager1=new Manager("Chin",6000);
7         manager1.add(emp1);
8         manager1.add(emp2);
9         Employee emp3=new Developer("Khair", 4500);
10        Manager generalManager=new Manager("Tan", 10000);
11        generalManager.add(emp3);
12        generalManager.add(manager1);
13        generalManager.print();
14    }
15 }
```



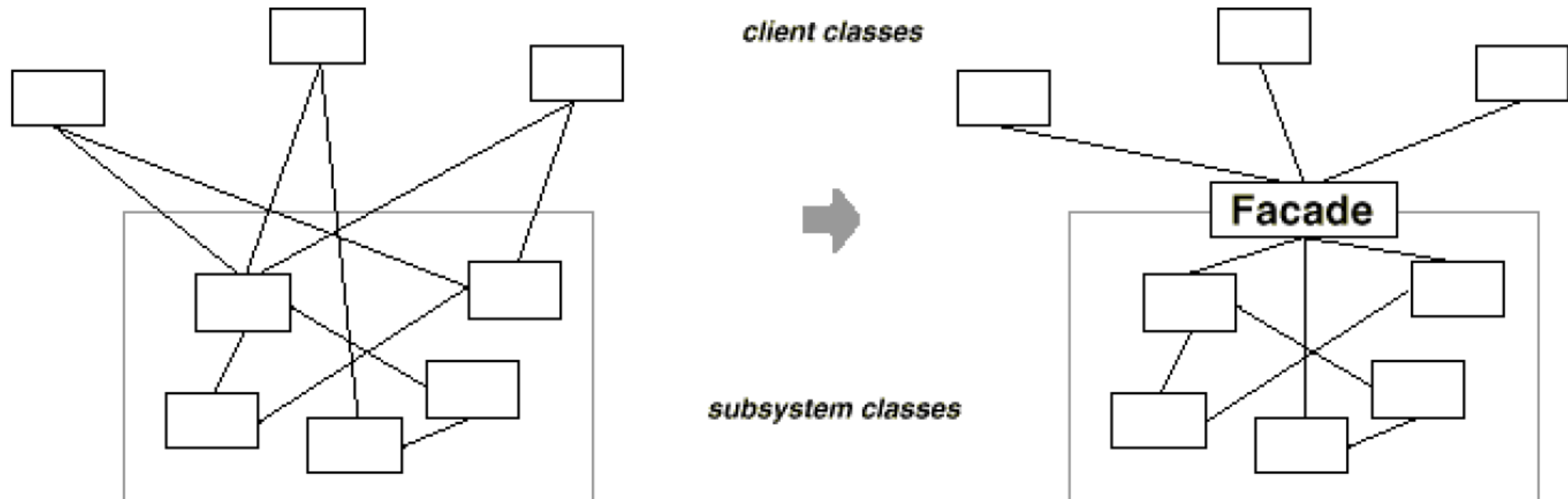
Composite : Structural Pattern (Example)

```
-----|  
Name =Tan  
Salary =10000.0  
-----  
-----  
Name =Khair  
Salary =4500.0  
-----  
-----  
Name =Chin  
Salary =6000.0  
-----  
-----  
Name =Mohammad  
Salary =3500.0  
-----  
-----  
Name =Lee  
Salary =4000.0  
-----
```



Façade : Structural Pattern

- **Intent:** Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.
- **Motivation(1):**
 - Structuring a system into subsystems helps reduce complexity. A common design goal is to minimize the communication and dependencies between subsystems. One way to achieve this goal is to introduce a **facade** object that provides a single, simplified interface to the more general facilities of a subsystem.



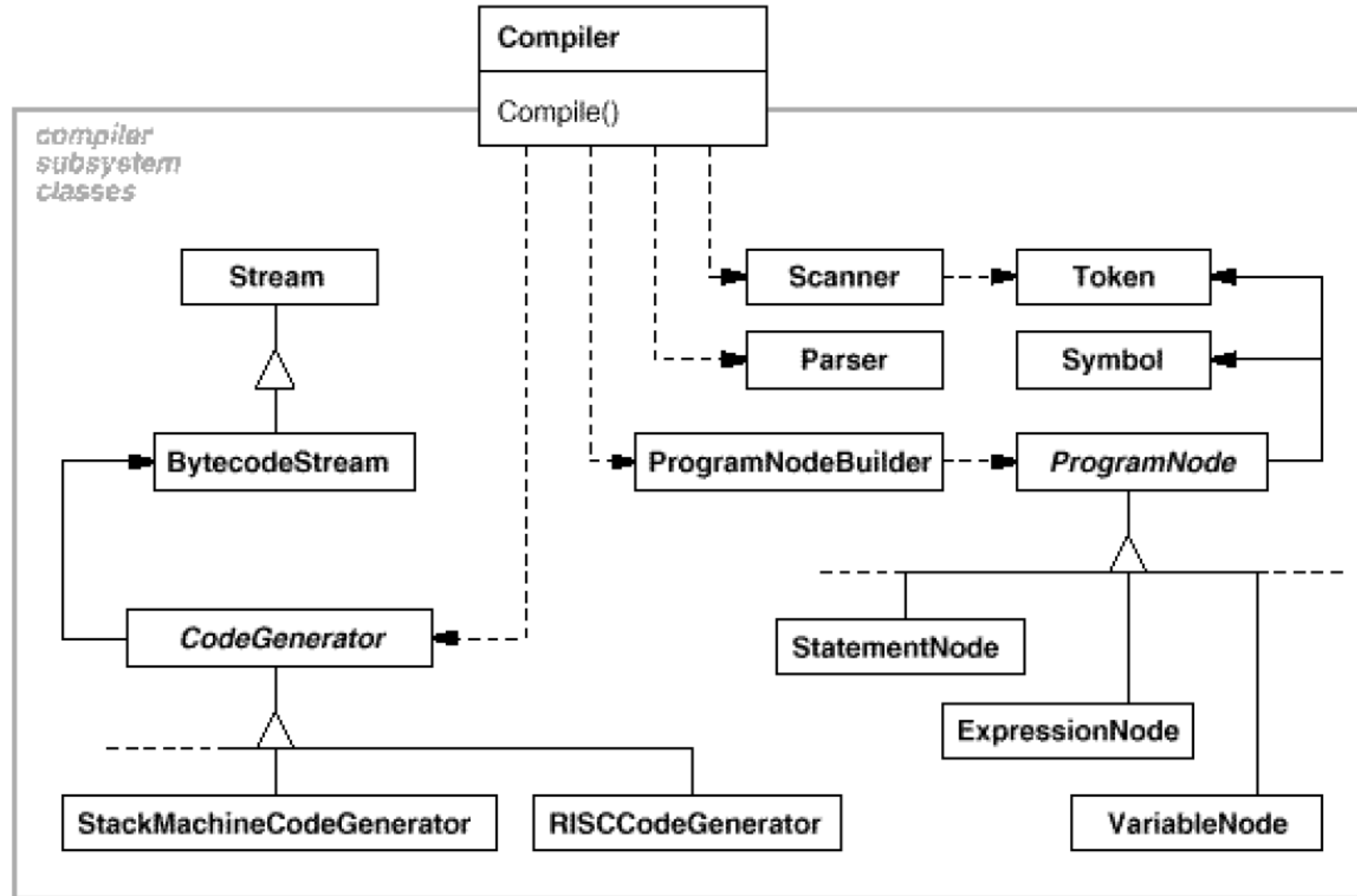
Façade : Structural Pattern

- **Motivation(2):**
 - Consider for example a programming environment that gives applications access to its compiler subsystem.
 - This subsystem contains classes such as Scanner, Parser, ProgramNode, BytecodeStream, and ProgramNodeBuilder that implement the compiler.
 - Some specialized applications might need to access these classes directly. But most clients of a compiler generally don't care about details like parsing and code generation; they merely want to compile some code.
 - For them, the powerful but low-level interfaces in the compiler subsystem only complicate their task..

Façade : Structural Pattern

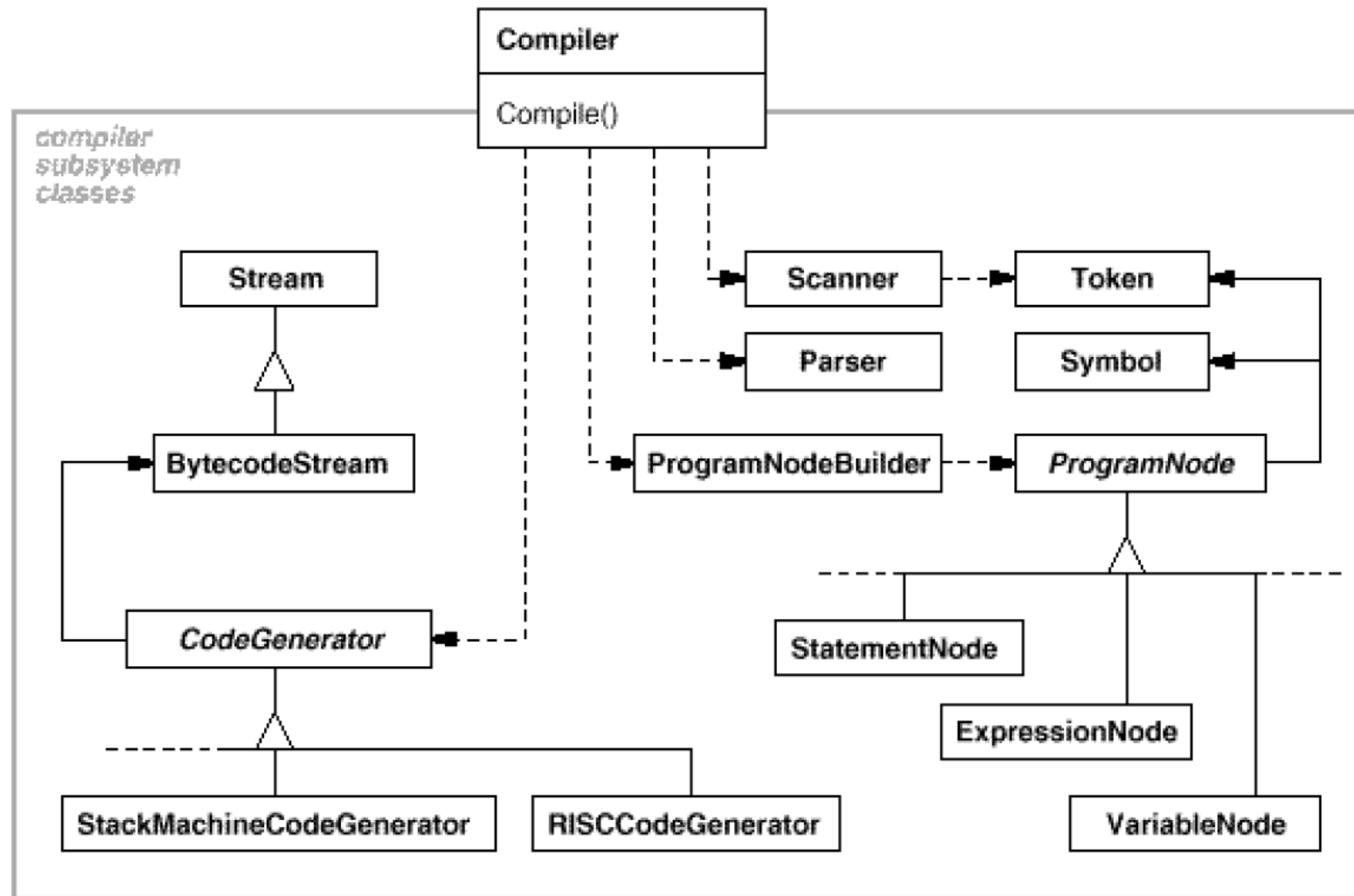
- **Motivation(3):**

- To provide a higher-level interface that can shield clients from these classes, the compiler subsystem also includes a **Compiler** class.
- This class defines a unified interface to the compiler's functionality. The **Compiler** class acts as a **façade**: It offers clients a single, simple interface to the compiler subsystem.
 - It **glues** together the classes that **implement** compiler functionality without hiding them completely.



Façade : Structural Pattern

- **Motivation(4):**
 - The **compiler facade** makes life easier for most programmers **without hiding** the **lower-level functionality** from the few that need it.

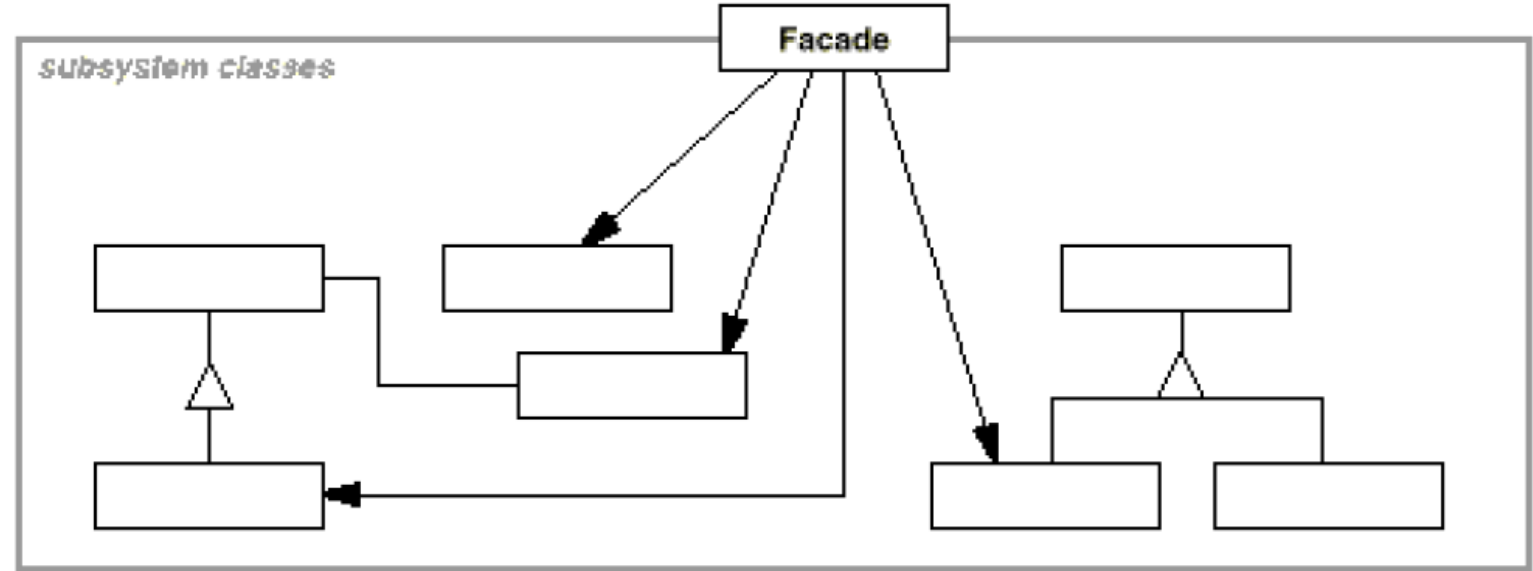


Façade : Structural Pattern

- **Applicability:** Use the Facade pattern when
 - you want to **provide** a **simple interface** to a **complex subsystem**. Subsystems often get more complex as they evolve. Most patterns, when applied, result in more and smaller classes. This makes the subsystem more reusable and easier to customize, but it also becomes harder to use for clients that don't need to customize it. A facade can provide a simple default view of the subsystem that is good enough for most clients. Only clients needing more customizability will need to look beyond the facade.
 - there are **many dependencies between clients** and the **implementation classes** of an abstraction. **Introduce** a **facade** to **decouple** the **subsystem from clients** and other **subsystems**, thereby **promoting** subsystem **independence** and **portability**.
 - you want to **layer** your **subsystems**. **Use** a **facade** to **define** an **entry point** to each **subsystem level**. If subsystems are dependent, then you can simplify the dependencies between them by making them communicate with each other solely through their facades.

Façade : Structural Pattern

- **Structure:**



- **Participants:**

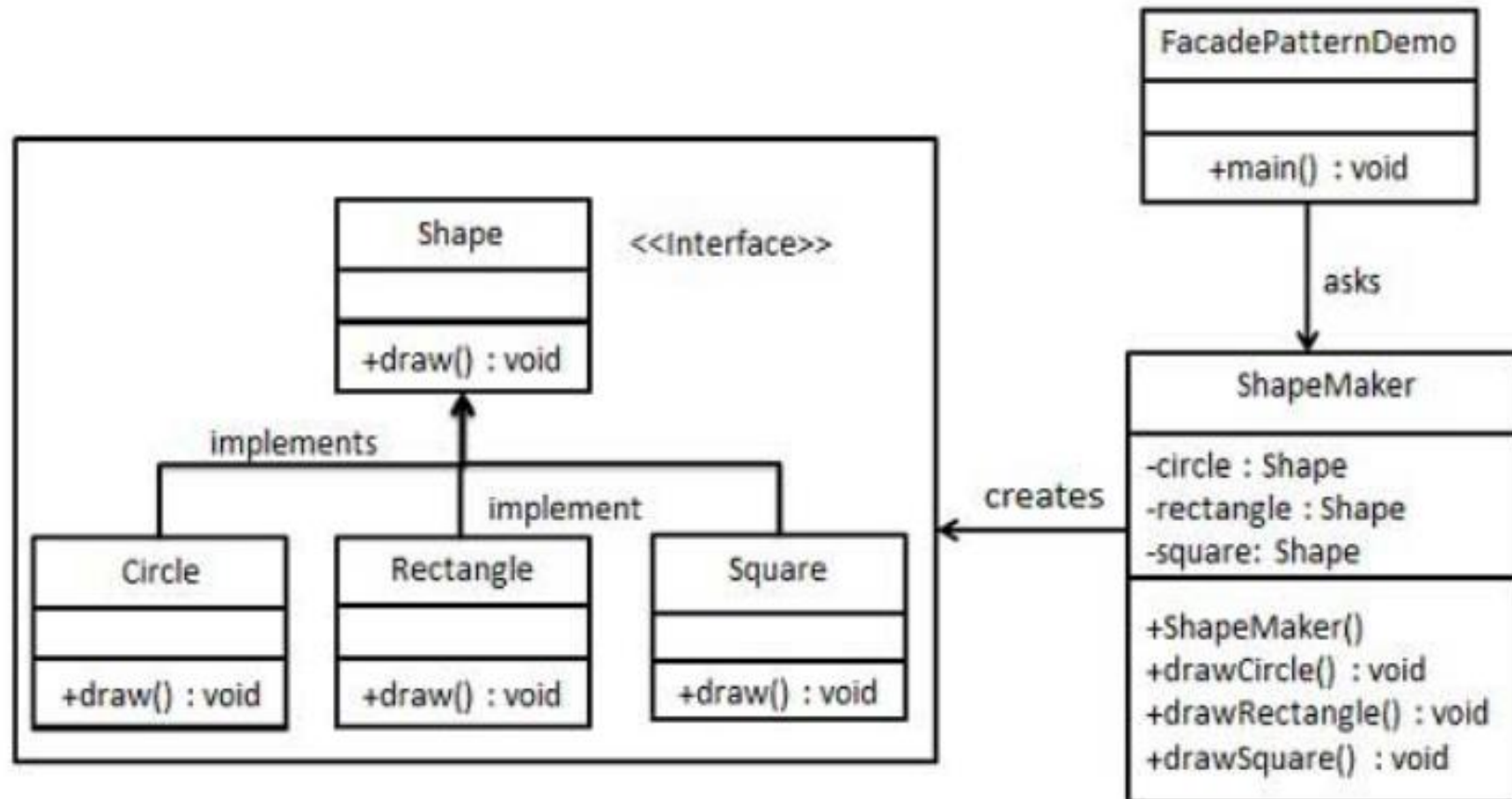
- Facade (Compiler)
 - knows which subsystem classes are responsible for a request.
 - delegates client requests to appropriate subsystem objects.
- subsystem classes (Scanner, Parser, ProgramNode, etc.)
 - implement subsystem functionality.
 - handle work assigned by the Facade object.
 - have no knowledge of the facade; that is, they keep no references to it.

Façade : Structural Pattern

- **Collaborations:**
 - Clients communicate with the subsystem by sending requests to Facade, which forwards them to the appropriate subsystem object(s). Although the subsystem objects perform the actual work, the facade may have to do work of its own to translate its interface to subsystem interfaces.
 - Clients that use the facade don't have to access its subsystem objects directly.

Façade : Structural Pattern (Example)

- We are going to create a Shape interface and concrete classes implementing the Shape interface. A facade class ShapeMaker is defined as a next step.
- ShapeMaker class uses the concrete classes to delegate user calls to these classes. FacadePatternDemo, our demo class, will use ShapeMaker class to show the results

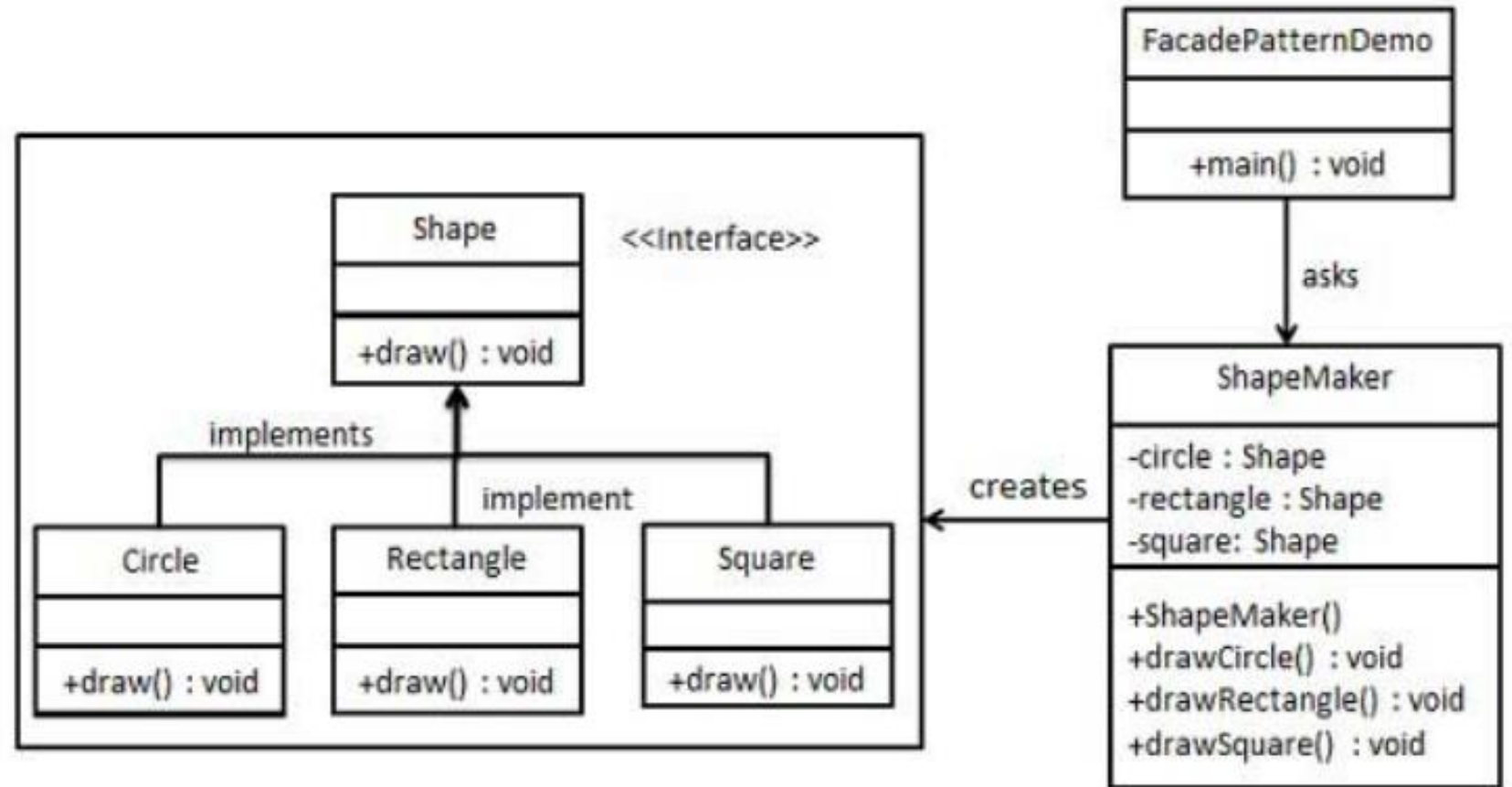


Façade : Structural Pattern (Example)

Create an interface.

Shape.java

```
public interface Shape {  
    void draw();  
}
```

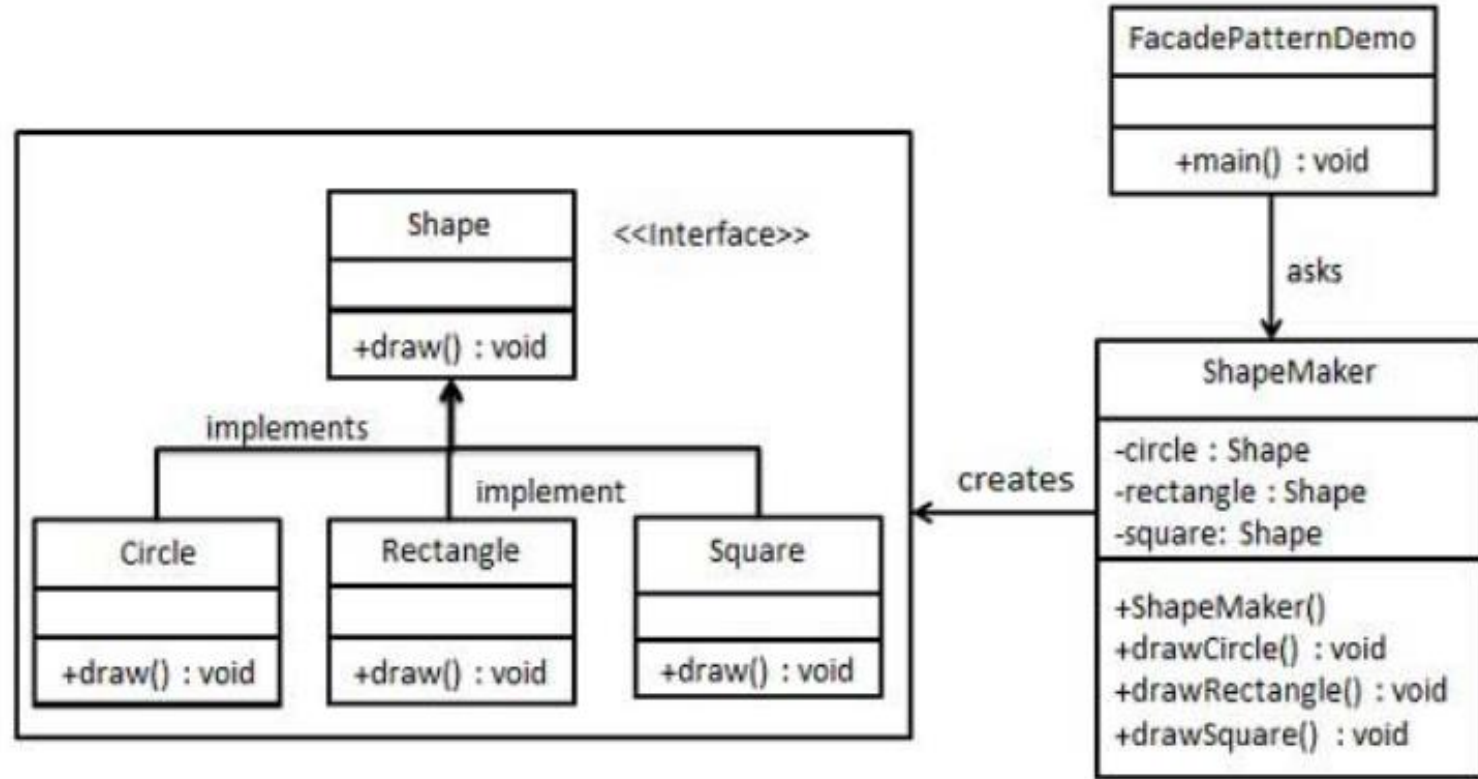


Façade : Structural Pattern (Example)

Create concrete classes implementing the same interface.

Rectangle.java

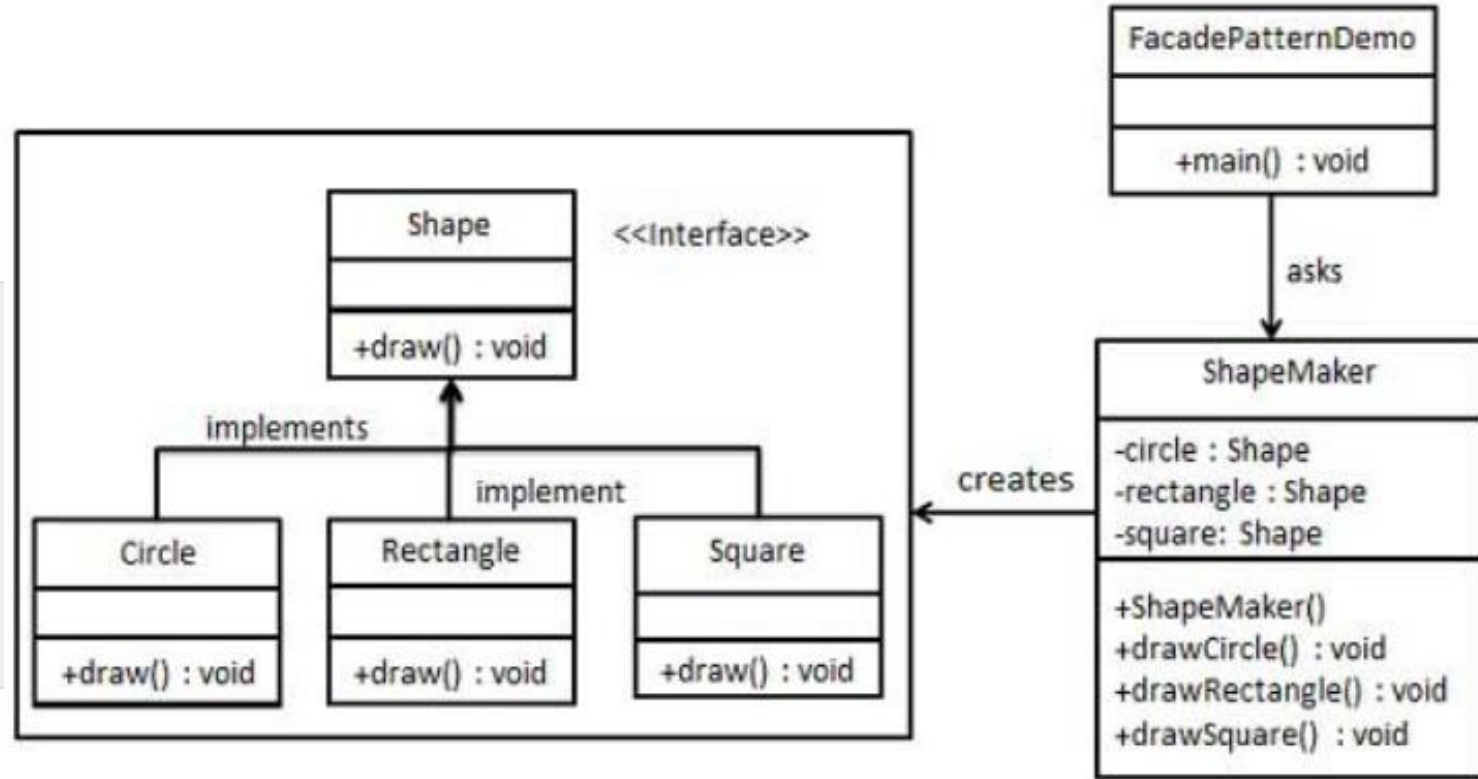
```
public class Rectangle implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Rectangle::draw()");  
    }  
}
```



Façade : Structural Pattern (Example)

Square.java

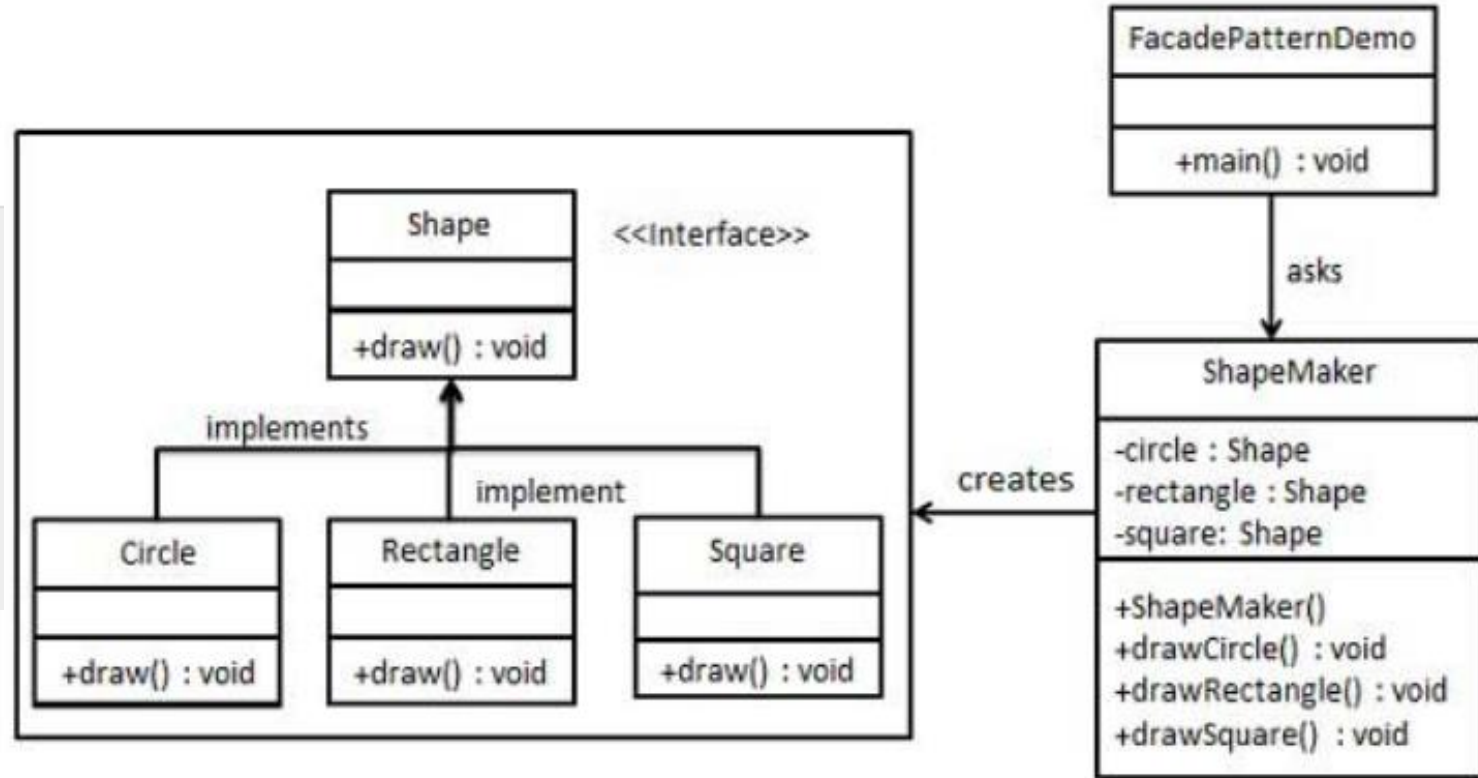
```
public class Square implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Square::draw()");  
    }  
}
```



Façade : Structural Pattern (Example)

Circle.java

```
public class Circle implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Circle::draw()");  
    }  
}
```

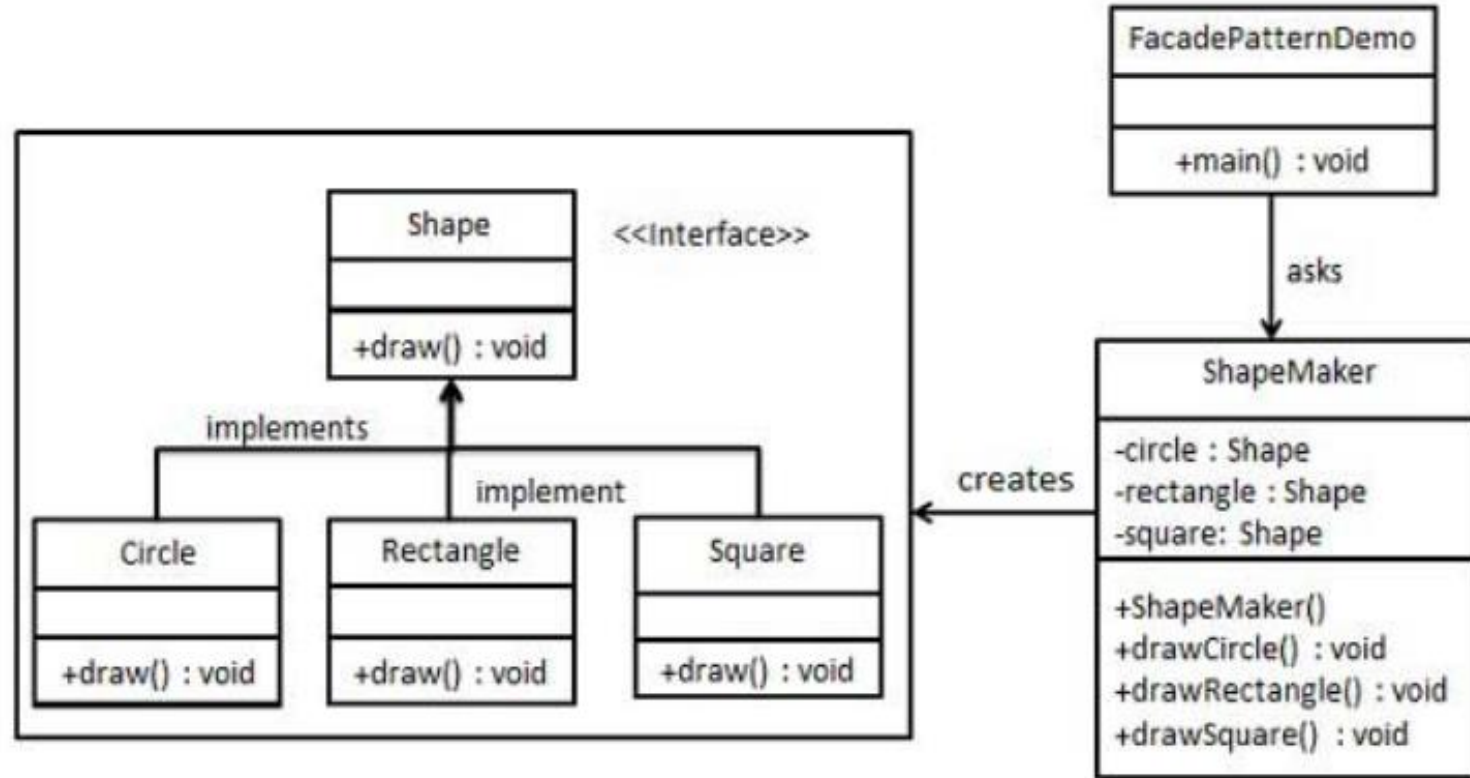


Façade : Structural Pattern (Example)

Create a facade class.

ShapeMaker.java

```
public class ShapeMaker {  
    private Shape circle;  
    private Shape rectangle;  
    private Shape square;  
  
    public ShapeMaker() {  
        circle = new Circle();  
        rectangle = new Rectangle();  
        square = new Square();  
    }  
  
    public void drawCircle() {  
        circle.draw();  
    }  
    public void drawRectangle() {  
        rectangle.draw();  
    }  
    public void drawSquare() {  
        square.draw();  
    }  
}
```

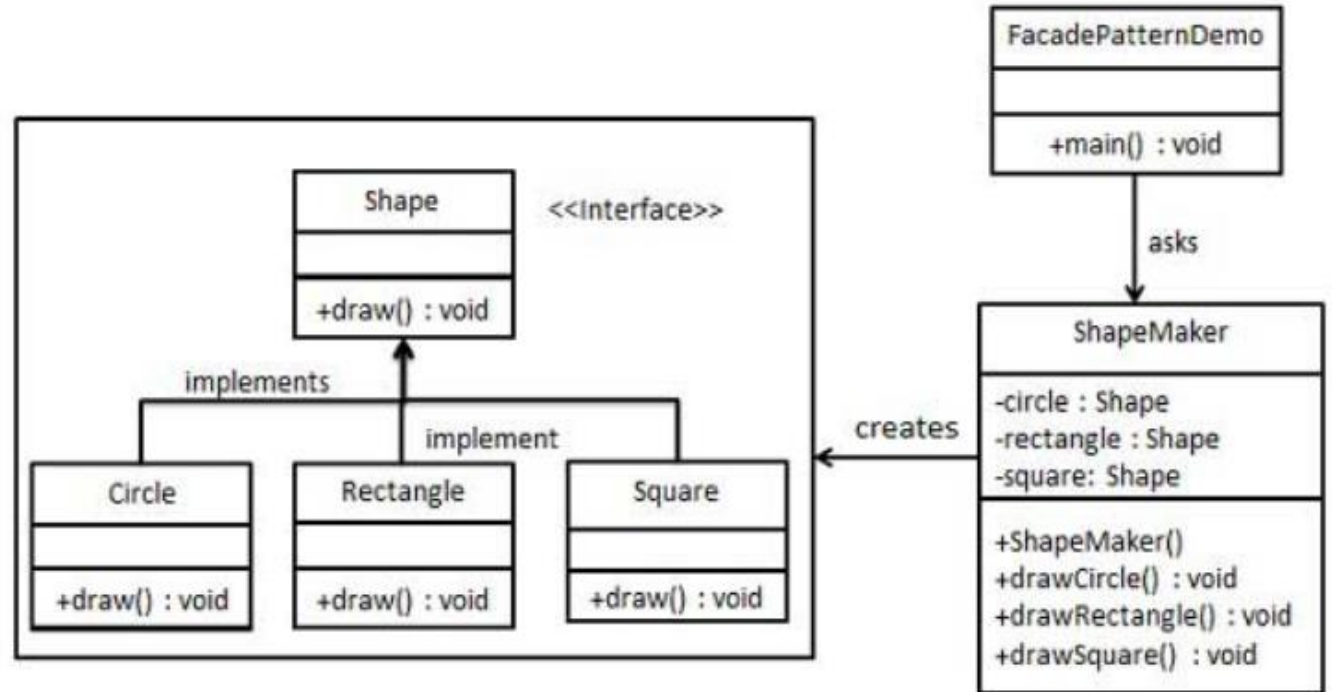


Façade : Structural Pattern (Example)

Use the facade to draw various types of shapes.

FacadePatternDemo.java

```
public class FacadePatternDemo {  
    public static void main(String[] args) {  
        ShapeMaker shapeMaker = new ShapeMaker();  
  
        shapeMaker.drawCircle();  
        shapeMaker.drawRectangle();  
        shapeMaker.drawSquare();  
    }  
}
```



Verify the output.

```
Circle::draw()  
Rectangle::draw()  
Square::draw()
```